

# AA3 - Informe Analítico sobre Vulnerabilidades

## Integrantes:

- Ariana Sarango
- Marco Chilliguano
- Emilio Jaramillo

Fecha: 19/10/2025

Proyecto: **MiniWorks**

Descripción general:

- MiniWorks es una plataforma que conecta a pequeños empleadores locales con estudiantes de la Universidad Nacional de Loja (UNL) para la realización de trabajos de corta duración (por horas o por día).
- El presente informe analiza las principales vulnerabilidades de seguridad que podrían afectar la aplicación, siguiendo el estándar OWASP Top 10 (2021)

## Entorno de Desarrollo

Se utiliza un **entorno basado en contenedores** para garantizar portabilidad, consistencia y escalabilidad.

- **Contenerización:** Docker. Cada microservicio, la base de datos y la aplicación web se empaquetan en contenedores individuales.
- **Control de Versiones:** Git, con repositorio en GitHub para control de código fuente y colaboración.

## Detalle de Componentes: Lenguajes y Frameworks

### Backend (Microservicios)

- **Lenguaje:** JavaScript (Node.js 20+)
- **Framework:** Express.js. Elegido por su flexibilidad y amplio soporte en APIs RESTful.
- **Base de Datos:** MongoDB (Atlas). Cada microservicio puede compartir la base de datos según la necesidad, usando colecciones específicas para mantener la modularidad de datos.
- **Protocolo de Comunicación:** API RESTful, utilizando **JSON** como formato de intercambio de información.
- **Autenticación:** JWT (JSON Web Tokens) para proteger rutas y gestionar sesiones de usuarios y gerentes.

### Frontend - Aplicación Web

- **Lenguaje (Cliente):** HTML, CSS, JavaScript
- **Framework (Opcional):** React para interfaces dinámicas y reactivas.
- **Backend-for-Frontend (BFF):** FastAPI no se usa aquí; la aplicación web consume directamente los microservicios vía REST y gestiona sesiones a través del backend de Node.js.

### Aplicación Móvil (Estudiantes y Empleadores)

- **Lenguaje:** React Native
- **Función:** Consumir los microservicios de MiniWorks, permitir registro, búsqueda de empleos, postulación y gestión de perfil.

### Servicios Adicionales

- **Documentación API:** Swagger/OpenAPI para documentar automáticamente los endpoints de cada microservicio.
- **Gestión de Dependencias:** npm (Node.js) y pip (Python, solo si se integra algún servicio con scripts en Python).

### A01: Broken Access Control

El control de acceso roto ocurre cuando un atacante puede acceder o modificar recursos sin tener los permisos adecuados.

Riesgo detectado:

Las rutas `/api/auth` y `/api/users` permiten operaciones críticas que podrían ser accedidas por usuarios sin los permisos adecuados.

Por ejemplo: un estudiante podría intentar publicar un trabajo si el backend no valida su rol.

Mitigación:

Se implementó un middleware `protect()` que valida el token JWT y el rol del usuario antes de el acceso.

### A02: Cryptographic Failures

Esta vulnerabilidad se da cuando la aplicación maneja mal datos sensibles (como contraseñas o tokens), los guarda en texto plano o usa algoritmos débiles.

Riesgo detectado:

Si las contraseñas se almacenan en texto plano, un atacante que acceda a la base de datos podría obtener todas las credenciales.

Mitigación:

Antes de guardar una contraseña, se usa `bcrypt` para generar un **hash seguro**

### A05: Security Misconfiguration

Errores en la configuración del servidor pueden exponer información sensible o permitir ataques directos.

**Riesgo detectado:**

Exposición accidental de variables sensibles (.env) o configuración insegura de CORS.

**Mitigación:**

- Configuración de CORS solo para dominios de confianza.
- No incluir .env en el repositorio Git.

## **A06 – Vulnerable and Outdated Components**

El uso de versiones obsoletas de dependencias puede introducir vulnerabilidades conocidas.

- Actualización de dependencias críticas (mongoose, jsonwebtoken, bcrypt, helmet).
- Documentación en README .md de las versiones seguras mínimas requeridas.

## **A07 – Identification and Authentication Failures (manejo incorrecto del token)**

Esta vulnerabilidad ocurre cuando el sistema no maneja correctamente los mecanismos de autenticación, lo que podría permitir acceso no autorizado o suplantación de identidad.

**Riesgo detectado:**

Si el token JWT no se valida adecuadamente o no expira correctamente, un atacante podría reutilizar un token robado o manipulado para acceder a información protegida. Además, la falta de revocación de tokens podría permitir el acceso incluso después de cerrar sesión.

**Mitigación:**

- Validación del token JWT en cada solicitud a rutas protegidas mediante el middleware protect().
- Configuración de tiempo de expiración corto para los tokens (por ejemplo, 1 hora).
- Implementación de una lista negra (blacklist) de tokens para invalidar sesiones activas tras el cierre de sesión o cambios de credenciales.
- Uso de https para evitar la interceptación de tokens en tránsito.

Esta vulnerabilidad ocurre cuando el sistema no maneja correctamente los mecanismos de autenticación, lo que podría permitir acceso no autorizado o suplantación de identidad.

**Riesgo detectado:**

**Mitigación:**

- ## A01: Broken Access Control

The screenshot displays the Insomnia application window. The top menu bar includes 'Application', 'File', 'Edit', 'View', 'Window', 'Tools', and 'Help'. Below the menu is a search bar and a 'Run' button. The main workspace is divided into three panes. The left pane shows a collection named 'My Collection' with a 'Run' button and a list of items: 'Base Environment', 'Add Cookies', 'Add Certificates', and a 'Filter' input. The middle pane shows a 'POST' request to 'http://localhost:3000/api/admin/auth/register'. The 'Body' tab is selected, showing a JSON payload: 

```
{ 1: { 2: "name": "Carlos Perez", 3: "email": "carlos.perez@empresa.com", 4: "password": "MiContraseñaSegurísima", 5: "role": "PUBLICADOR DE TRABAJO" 6: } }
```

. The right pane shows the 'Preview' of the response, which is a JSON object: 

```
{ 1: { 2: "message": "Usuario registrado con éxito.", 3: "id": "6d75b44ee3c7e2ab7e10e22" 4: } }
```

. The top right of the interface shows a 'Send' button, a status bar indicating '201 Created', '264 ms', and '76 B', and a user profile for 'marco.chiligrano@unl.edu.ec'.

```
src > modules > auth > authService.js > AuthService
1  const bcrypt = require('bcrypt');    File is a CommonJS module; it may be converted to an ES module.
2  const userRepository = require('../../shared/repositories/user.repository');
3  class AuthService {
4      async login(email, password) {
5          const user = await userRepository.findByEmail(email);
6
7          if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
8              throw { status: 401, message: "Credenciales inválidas." };
9          }
10
11         return {      id: user.id,
12                     name: user.name,
13                     email: user.email,
14                     rol: user.rol };
15     }
16 }
17
18
19 module.exports = new AuthService();
```

y aqui aplicamos el JWT



```

src > middlewares > JS auth.middleware.js > ...
1  const jwt = require('jsonwebtoken');
2  require('dotenv').config();
3
4  function protect(req, res, next) {
5    try {
6      const authHeader = req.headers.authorization;
7
8      if (!authHeader || !authHeader.startsWith('Bearer ')) {
9        return res.status(401).json({ message: 'Token no proporcionado.' });
10     }
11
12     const token = authHeader.split(' ')[1];
13     const decoded = jwt.verify(token, process.env.JWT_SECRET);
14
15     req.user = decoded;
16
17     next();
18   } catch (error) {
19     res.status(401).json({ message: 'Token inválido o expirado.' });
20   }
21 }
22
23 module.exports = { protect };
24

```

Aplicamos para q el token de seguridad se va a validar por un tiempo

```

1  JWT_SECRET=este es un secreto muy difícil de adivinar 1234
2  JWT_EXPIRES_IN=1h

```

## A06 – Vulnerable and Outdated Components

```

### 2. Configurar Backend (Express.js)
cd backend
npm install
cp .env.example .env # Configurar variables de entorno (MongoDB URI, JWT_SECRET, etc.)
npm run dev

```

Explicamos en el readme de nuestro repositorio las versiones seguras mínimas requeridas.

### Aplicaciones Futuras:

## Frontend

### A03 – Injection (XSS, HTML Injection)

Las vulnerabilidades de inyección ocurren cuando la aplicación permite que datos no validados del usuario se interpreten como parte del código ejecutable o HTML, lo que puede permitir ataques como Cross-Site Scripting (XSS).

#### Riesgo detectado:

Los campos de entrada en formularios (por ejemplo, título y descripción de trabajo) podrían permitir la inserción de código HTML o scripts maliciosos si no se sanitizan adecuadamente.

Un atacante podría inyectar un <script> en una publicación, lo que ejecutaría código malicioso en el navegador de otros usuarios.

**Mitigación:**

- Sanitización de todas las entradas del usuario en el backend antes de almacenarlas en la base de datos.
- Implementación de la librería express-validator o sanitize-html para limpiar campos de texto.
- Uso de helmet con configuración para prevenir inyección de scripts (por ejemplo, contentSecurityPolicy).
- Escapado automático de datos al renderizar información en el frontend.

**A08 – Software and Data Integrity Failures (uso de librerías inseguras en el frontend)**

Este tipo de falla ocurre cuando el sistema confía en componentes, scripts o librerías externas sin verificar su integridad o procedencia.

**Riesgo detectado:**

El frontend de MiniWorks podría depender de librerías externas (como Bootstrap, React o Axios) cargadas desde CDNs sin verificación de integridad. Si una de estas fuentes es comprometida, los atacantes podrían inyectar código malicioso directamente en la aplicación.

**Mitigación:**

- Uso de versiones específicas y verificadas de librerías en el archivo package.json.
- Evitar el uso de CDNs sin el atributo integrity y crossorigin.
- Implementación de auditorías periódicas con npm audit para detectar dependencias inseguras.
- Automatización de actualizaciones críticas con herramientas como npm-check-updates.

**APP MÓVIL**

**A04 – Insecure Design (falta de validación o restricciones en cliente)**

Las fallas de diseño inseguro ocurren cuando la aplicación no implementa controles adecuados en su arquitectura o lógica de negocio, lo que permite a usuarios realizar acciones fuera de su flujo legítimo. No se trata de errores de implementación, sino de ausencia de validaciones o restricciones desde la etapa de diseño.

**Riesgo detectado en MiniWorks:**

En la app móvil y frontend, la validación de acciones depende parcialmente del lado del cliente. Por ejemplo, el cliente podría modificar peticiones para publicar trabajos o acceder a datos restringidos si no se aplican controles en el backend.

Un atacante con herramientas como Postman o Burp Suite podría enviar peticiones alteradas directamente a la API (/api/jobs, /api/users) sin usar la interfaz oficial, eludiendo validaciones de la app.

#### **Mitigación:**

- Implementar validaciones y controles de acceso exclusivamente en el backend (no confiar en el cliente).
- Diseñar las rutas con principio de menor privilegio: cada rol (estudiante, publicador, administrador) solo puede acceder a las operaciones definidas para él.
- Establecer reglas de negocio claras: por ejemplo, un estudiante no puede crear trabajos, un publicador no puede postularse

### **A09 – Security Logging and Monitoring Failures**

La falta de registro y monitoreo impide detectar accesos no autorizados, errores de seguridad o ataques en curso. Sin logs adecuados, los incidentes pueden pasar desapercibidos o no dejar evidencia para su análisis.

#### **Riesgo detectado en MiniWorks:**

Actualmente, el backend podría no registrar eventos de seguridad importantes (por ejemplo: intentos fallidos de inicio de sesión, accesos denegados, manipulación de tokens o errores 500).

Además, si no existen alertas o revisiones periódicas de los logs, un atacante podría realizar múltiples intentos de fuerza bruta o explorar rutas protegidas sin ser detectado.

#### **Mitigación:**

- Implementar registro estructurado en el backend usando librerías como winston o morgan.
- Registrar eventos clave: inicios de sesión fallidos, tokens inválidos, cambios de roles, creación/eliminación de trabajos y accesos denegados.
- Almacenar logs en un servicio centralizado (por ejemplo, Winston + MongoDB o Elastic Stack).
- Evitar incluir datos sensibles (contraseñas, tokens JWT) en los logs.

### **A10 – Server-Side Request Forgery (SSRF)**

SSRF ocurre cuando una aplicación (o API) permite que un usuario fuerce al servidor a realizar solicitudes HTTP a recursos internos o externos arbitrarios.



Esto puede permitir que el atacante acceda a endpoints internos, metaservicios (como AWS metadata), o realice escaneos desde el servidor.

### **Riesgo detectado en MiniWorks:**

Si algún endpoint del backend (por ejemplo, uno para subir imágenes, validar URLs o integrar servicios externos como correo o almacenamiento en la nube) permite al usuario enviar una URL sin validación, un atacante podría hacer que el servidor realice peticiones a destinos internos (http://localhost, 169.254.169.254, etc.).

Esto expondría servicios internos de la infraestructura o datos sensibles.

### **Mitigación:**

- Nunca permitir que el usuario envíe directamente URLs que el servidor consultará. Si es necesario (por ejemplo, validación de enlaces), filtrar y validar que el dominio sea de confianza.
- Restringir salidas HTTP desde el servidor (firewall o reglas de salida).
- Verificar protocolos (solo https:// permitido), longitud y formato de URLs.

Aplicación De medidas de seguridad:

### **1. Vulnerabilidad principal:**

**Broken Authentication**, porque usuarios no autorizados podrían acceder a información sensible o manipular datos.

### **2. Validación de mitigación:**

Pruebas de acceso con tokens válidos e inválidos, expiración de JWT, roles correctos y contraseñas hasheadas.

### **3. Relación con C4:**

- **API Gateway:** controla autenticación y accesos.
- **Backend:** valida roles y entradas, evita inyecciones.
- **Base de datos:** protege datos sensibles y previene inyecciones NoSQL/SQL.

### **Reflexión de lo aprendido:**

Durante el desarrollo de MiniWorks, he aprendido a integrar múltiples tecnologías y enfoques arquitectónicos para construir un sistema robusto y escalable. La implementación de una arquitectura de microservicios me permitió comprender la importancia de desacoplar responsabilidades, facilitando la mantenibilidad y la escalabilidad del proyecto.

Trabajar con Node.js, Express y MongoDB reforzó mis conocimientos en desarrollo de APIs RESTful, gestión de datos y autenticación mediante JWT, mientras que la integración de React Native y la aplicación web me enseñó a construir interfaces dinámicas y amigables

para el usuario final. Además, el uso de Docker y control de versiones con Git/GitHub me permitió apreciar la relevancia de la portabilidad y la colaboración en proyectos de software.